

Functions

Methods (Functions)

```
static void Main(string[] args)
{
    int num1 = 1;
    int num2 = 2;
    int sum = num1 + num2;
    Console.WriteLine($"The sum of {num1} and {num2} is {sum}.");

    num1 = 5;
    num2 = 9;
    sum = num1 + num2;
    Console.WriteLine($"The sum of {num1} and {num2} is {sum}.");

    num1 = 2;
    num2 = 7;
    sum = num1 + num2;
    Console.WriteLine($"The sum of {num1} and {num2} is {sum}.");
}
```

```
static void Main(string[] args)
{
    PrintSum(1, 2);
    PrintSum(5, 9);
    PrintSum(2, 7);
}

static void PrintSum(int num1, int num2)
{
    int sum = num1 + num2;
    Console.WriteLine($"The sum of {num1} and {num2} is {sum}.");
}
```

Methods (Functions)

Modifier Return Type Function name Parameter list

↓ ↓ ↓ ↓

```
static int Add(int a, int b)
{
    int c = a + b;
    return c;
}
```

Return value and function completion

Call:

```
int sum = Add(1, 2);
```

Methods (Functions)

- Modifiers define the access options to or the state of the function.

e.g.: `static`
`public`
`private`

- The return value specifies the data type of the return

e.g.: `int`
`Random`
`void`

- The function is called via the identifier

- The parameter list specifies the required transfer parameters with data type and identifier

Method overloading

- a function name can be used several times in the same class
- each function must have a unique parameter list
- the return values can be different

```
static int Add(int a, int b)
{
    int c = a + b;
    return c;
}
```

```
static int Add(int a, int b, int c)
{
    int c = a + b;
    return c;
}
```

Special parameter types

- Automatic arrays: An array is automatically created from any number of transfers of a type.

```
static int SumNumbers(params int[] numbers) {...}
```

Call examples: `SumNumbers(2,5,7);` `SumNumbers(1,2,3,4,5,6,7)`

- Optional parameters: A parameter is assigned a default value which can be overwritten.

```
public static int Add(int a, int b, int c = 0) {...}
```

Call examples: `Add(2,5);` `Add(1,2,3)`

Special parameter types - out

- a function can always have only one return value
- more than one value can be returned using out
- Parameter is declared in the signature and must be initialized in the function
- the call is also made with the keyword

```
int Calculate(int num1, int num2, out int min, out int max, out int avg)
{
    min = Math.Min(num1, num2);
    max = Math.Max(num1, num2);
    avg = (num1 + num2) / 2;
    return num1 + num2;
}
```

```
int min;
int max;
int avg;
int sum = Calculate(3, 5, out min, out max, out avg);
```

```
//sum = 8
//min = 3
//max = 5
//avg = 4
```